

# **HOTEL MANAGEMENT SYSTEM**

## **Project Report**

VITyarthi – Build Your Own Project

Developer: Yatharth Shukla

Registration Number: 25BAI10751

Institution: VIT Bhopal University

Program: B.Tech Artificial Intelligence and Machine Learning

Academic Year: 2026–27

## 1. Introduction

The Hotel Management System (HMS) is a Java-based application designed to organize essential hotel operations through a centralized workflow. The system focuses on guest management, room management, reservations, check-in/check-out and billing. It demonstrates the application of programming and software-development concepts to a practical real-world problem.

## 2. Problem Statement

Hotels need to manage a large amount of information related to guests, rooms, reservations and bills. When these activities are maintained through separate manual records or repetitive data entry, retrieving information and keeping records consistent can become difficult. The Hotel Management System provides a structured software solution for bringing these core activities into one application.

## 3. Objectives

- To maintain guest/customer information in an organized manner.
- To manage room details and room availability.
- To create, modify, search and cancel reservations.
- To support check-in and check-out operations.
- To calculate and generate hotel bills.
- To reduce repetitive manual record-management work.
- To provide input validation and meaningful error handling.
- To demonstrate modular and maintainable Java implementation.

## 4. Scope of the Project

The scope includes guest registration and record management, room management, reservation operations, check-in/check-out, billing, search/update functionality, validation and basic reporting. Advanced integrations such as real payment gateways, online travel-platform synchronization and multi-branch cloud deployment are considered future enhancements unless implemented in the submitted version.

## 5. Target Users

- Hotel receptionist/front-desk staff
- Hotel administrator
- Reservation staff
- Billing/front-office staff

## 6. Functional Requirements

FR1 – Guest Management: the system shall add, view, search, update and delete guest records.

FR2 – Room Management: the system shall maintain room numbers, types, pricing and availability/status.

FR3 – Reservation Management: the system shall create, modify and cancel reservations.

FR4 – Check-in/Check-out: the system shall maintain guest stay status and update room availability.

FR5 – Billing: the system shall calculate applicable charges and generate a bill.

FR6 – Search and Update: the system shall retrieve and modify existing records.

FR7 – Validation: the system shall validate required input and reject invalid operations.

FR8 – Reporting: the system shall provide organized views of relevant hotel records where implemented.

## 7. Non-Functional Requirements

- Usability – the application workflow should be simple and understandable.
- Performance – routine record operations should execute efficiently.
- Reliability – room and reservation information should remain consistent.
- Maintainability – functionality should be separated into logical modules/classes.
- Scalability – new modules should be addable without redesigning the entire system.
- Error Handling – invalid IDs, unavailable rooms and incorrect input should be handled gracefully.
- Resource Efficiency – unnecessary processing and duplicate data should be minimized.

## 8. Execution Environment

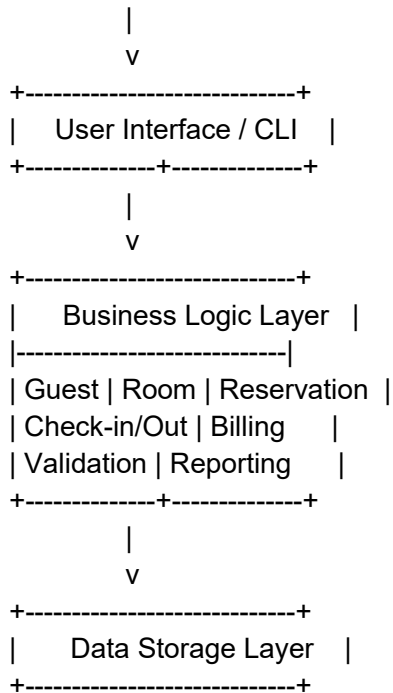
The intended implementation is 100% Core Java and can be executed from the command line without GUI dependencies, provided the submitted source code follows this design. The exact JDK version and any database/file-storage mechanism should be documented according to the actual implementation.

## 9. System Architecture

The system follows a modular layered structure. The user interacts with the interface layer. The request is passed to business-logic modules such as Guest, Room, Reservation, Check-in/Check-out and Billing. Data is then stored or retrieved using the persistence mechanism implemented by the project.

System Architecture:

```
+-----+
|   Hotel Staff/User   |
+-----+-----+
```



## 10. Major Functional Modules

### 10.1 Guest Management

Maintains guest details and supports creation, search, update and deletion of guest records.

### 10.2 Room Management

Maintains room number, room type, pricing and room availability/status.

### 10.3 Reservation Management

Handles reservation creation, modification, cancellation and availability checking.

### 10.4 Check-in / Check-out

Maintains the active stay lifecycle and updates room status.

### 10.5 Billing Management

Calculates implemented room/service charges and generates the final bill.

### 10.6 User Management

Provides user or administrator access management where authentication is implemented.

### 10.7 Validation and Error Handling

Checks inputs, IDs, room availability and business rules before processing.

### 10.8 Reporting / Record Viewing

Provides organized views or summaries of operational records where implemented.

## 11. Process Workflow

Start  
|  
Open Hotel Management System  
|  
Select Required Operation  
|  
Guest / Room / Reservation / Check-in / Check-out / Billing  
|  
Validate Input  
|  
Process Request  
|  
Store / Retrieve / Update Data  
|  
Display Result  
|  
Continue or Exit

## 12. Use Case Diagram – Description

The primary actor is the Hotel Staff/Administrator. Main use cases are Manage Guest, Manage Room, Manage Reservation, Check-in Guest, Check-out Guest, Generate Bill, Search Records and View Reports. Authentication can be represented as a prerequisite use case if implemented.

## 13. Class / Component Design

The suggested logical classes/components are Main, Guest, Room, Reservation, Billing, Payment, User, DataManager, Validation and Report. The final class diagram should match the actual source code submitted in the repository.

## 14. Sequence Flow – Reservation

Staff/User → Reservation Module: enter guest and room details  
Reservation Module → Room Module: check availability  
Room Module → Reservation Module: availability result  
Reservation Module → Data Storage: save reservation  
Reservation Module → Staff/User: confirmation / error message

## 15. Database / Storage Design

If persistent storage is implemented, the core entities may include Guest, Room, Reservation, Bill/Payment and User. A guest can be associated with reservations; a reservation is associated with a room; a completed stay can produce a billing/payment

record. The final ER diagram and schema must reflect the actual database or storage implementation.

## 16. Design Decisions and Rationale

- A modular design separates major hotel operations and improves maintainability.
- Validation is placed before processing to reduce invalid state changes.
- Reservation and room management are connected so room availability can be checked before booking.
- Check-in/check-out updates the operational state of rooms.
- Billing is separated from reservation logic to keep charge calculation organized.
- Git/GitHub provides version control and a structured submission history.

## 17. Implementation Details

The implementation should contain at least 5–10 meaningful modules/classes/files as required by the supplied VITyarthi project brief. Source code should be organized into appropriate folders or packages and should include comments/documentation, validation, error handling and testing. The final report should list only the technologies and features actually implemented.

## 18. Testing Approach

Testing covers normal and exceptional workflows including guest registration, room availability, reservation creation and cancellation, check-in/check-out, billing, record search/update and invalid input handling.

| Test Case          | Input / Condition              | Expected Result                                     |
|--------------------|--------------------------------|-----------------------------------------------------|
| Guest Registration | Valid guest details            | Guest record created                                |
| Room Availability  | Available room                 | Room can be reserved                                |
| Unavailable Room   | Occupied/reserved room         | Booking rejected                                    |
| Reservation        | Valid guest and available room | Reservation created                                 |
| Cancellation       | Valid reservation ID           | Reservation cancelled                               |
| Check-in           | Valid reservation              | Guest checked in and room status updated            |
| Check-out          | Active stay                    | Stay closed, room made available and bill generated |
| Billing            | Valid stay                     | Bill calculated according to implemented rules      |
| Search             | Existing record ID             | Correct record displayed                            |
| Invalid Input      | Invalid/missing data           | Validation/error message displayed                  |

## 19. Screenshots / Results

Insert actual screenshots from the implemented application in this section. Recommended evidence includes the main menu, guest registration, room availability, reservation creation, check-in/check-out, billing output, successful operations and validation/error messages.

## 20. Challenges Faced

- Designing a workflow that consistently connects guests, rooms and reservations.
- Maintaining correct room availability during reservation, check-in and check-out.
- Handling invalid IDs and user input.
- Keeping the implementation modular while integrating multiple functions.
- Testing both normal and exceptional user workflows.

## 21. Learnings and Key Takeaways

- Application of programming concepts to a real-world management problem.
- Improved understanding of modular programming and project organization.
- Practical use of CRUD operations and validation.
- Understanding the relationship between requirements, design, implementation and testing.
- Experience with Git/GitHub and project documentation.

## 22. Future Enhancements

- Online room booking and cancellation.
- Distributed/cloud database for multiple branches.
- Real payment-gateway integration.
- Role-based authentication and authorization.
- Email/SMS reservation notifications.
- Restaurant, housekeeping and inventory management.
- Analytics dashboard.
- Web and mobile interfaces.
- Cloud deployment.

## 23. Conclusion

The Hotel Management System provides a structured approach to managing essential hotel operations. By combining guest, room, reservation, check-in/check-out and billing modules with validation, testing and organized documentation, the project demonstrates how software can be applied to a practical real-world management problem.

## 24. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied project brief.
- Official documentation for the programming language, database and tools actually used in the implementation.
- Relevant course notes and laboratory/lecture material.

## 25. Appendix – Design Artefacts Checklist

- System Architecture Diagram
- Process Flow / Workflow Diagram
- Use Case Diagram
- Class / Component Diagram
- Sequence Diagram
- ER Diagram / Schema Design, if storage is used
- Screenshots of actual implementation
- Testing evidence